Epochs: a backward-compatible language evolution mechanism

Document #:	P1881R0
Date:	2019-10-06
Project:	Programming Language C++ Evolution Working Group Incubator (EWGI)
Reply-to:	Vittorio Romeo <vittorio.romeo@outlook.com (mailto:vittorio.romeo@outlook.com)>

Contents

1 Abstract

This paper proposes a mechanism to evolve the C++ language syntax while retaining backward and forward compatibility by adding an opt-in module-level switch to change the meaning of source code.

2 Revision History

None.

3 Motivation

One of the pillars of C++ is *backward compatibility*: new standards are designed to minimize the number of changes making existing code ill-formed or silently behave in a different way. Conservativeness likely led to the language's success and survival, but also introduces several major drawbacks:

— Obsolete and outdated constructs needlessly increase the breadth and complexity of the language, possibly leading to *analysis paralysis*[1]. Examples include: `typedef` (superseded by `using`), or `std::bind` (superseded by *lambda expressions*). Removing such constructs would lead to a more consistent, modern, and smaller C++, giving fewer superfluous choices to newcomers and experts alike.

— C++ becomes less attractive over time as a language to choose for new projects. While still being a pragmatic choice for work which depends on existing C++ code, it is getting harder to justify choosing C++ over more modern alternatives due to its poor teachability, pitfalls, unsafe defaults, and complexity.

— Readability and terseness is often sacrificed. An example is the `co_await` keyword, which could not be named `await`.

— Dangerous defaults and constructs, often originating from C, cannot be removed or altered. Examples include: *uninitialized variables*, *implicit conversions*, and *macros*. Requiring a more explicit syntax to access such constructs would greatly reduce the likelihood of mistakes and make the language more welcoming to newcomers.

— Despite the hard work of the committee, newer features sometimes have flaws that only became obvious after extensive user experience, which cannot then be fixed. This leads the committee to be overly conservative and prevents a user-driven feedback loop. The interactions between `std::initializer_list` and *uniform initialization* are an example of this issue.

— Writing safe and modern C++ can only be enforced by convention, static analysis, or compilers' quality of implementation. Non-controversial Core Guidelines such as C.128[2] should instead be enforced by the language itself.

— The *"simplification paradox"*: a number of papers, such as P0709[3], claim to simplify the language by introducing a new facility which aims to replace an existing one. While consistent use of the newly proposed features might simplify a program, it is undeniable that widening the language increases its complexity and the cognitive overhead of developers (now having even more options to choose from).

This paper proposes a mechanism which would solve all the problems listed above, while still allowing C++ to remain backward and forward compatible: **epochs**.

4 Example: *implicit conversions*

As a thought experiment, imagine consensus being achieved among the committee to forbid *implicit conversions* between *fundamental types* in C++23, due to them often being a source of bugs and readability issues. The committee agreed that any such conversion will now be ill-formed, and that casts should be used if desired.

Applying this change directly to the standard would result in a massive breakage of existing (often business-critical) code, which would prevent most organizations from migrating to the newest version of the language. Even if some source files could independently be recompiled with C++23, any header file inclusion introducing an implicit conversion (e.g. as part of a *template definition*) will block the migration.

The committee, however, devises a solution: adding a module-level switch that allows developers to opt into the new change to *implicit conversion* rules:

Before	After

`module ParticleMovement;`
 	
`export`
`void move(Particle&, float x, float y);`
 	
`void moveExample()`
`{`
`  Particle p{};`
 	
`  // OK`
`  move(p, 3.42, 2.49);`
 	
`}`
	`epoch 2023; // Module-level switch`
	`module ParticleMovement;`
	
	`export`
	`void move(Particle&, float x, float y);`
	
	`void moveExample()`
	`{`
	`  Particle p{};`
	
	`  // Compilation error`
	`  ~~move(p, 3.42, 2.49);~~`
	
	`  // OK, no implicit conversions`
	`  move(p, 3.42f, 2.49f);`
	`}`

The *epoch-declaration* `epoch 2023` specified before the *module-declaration* would make all the code in the *module purview* obey epoch 2023's rules. Modules targeting epoch 2023 must not contain any *implicit conversion* between *fundamental types*.

Not specifying an *epoch-declaration* results in the module not opting into any epoch-specific change.

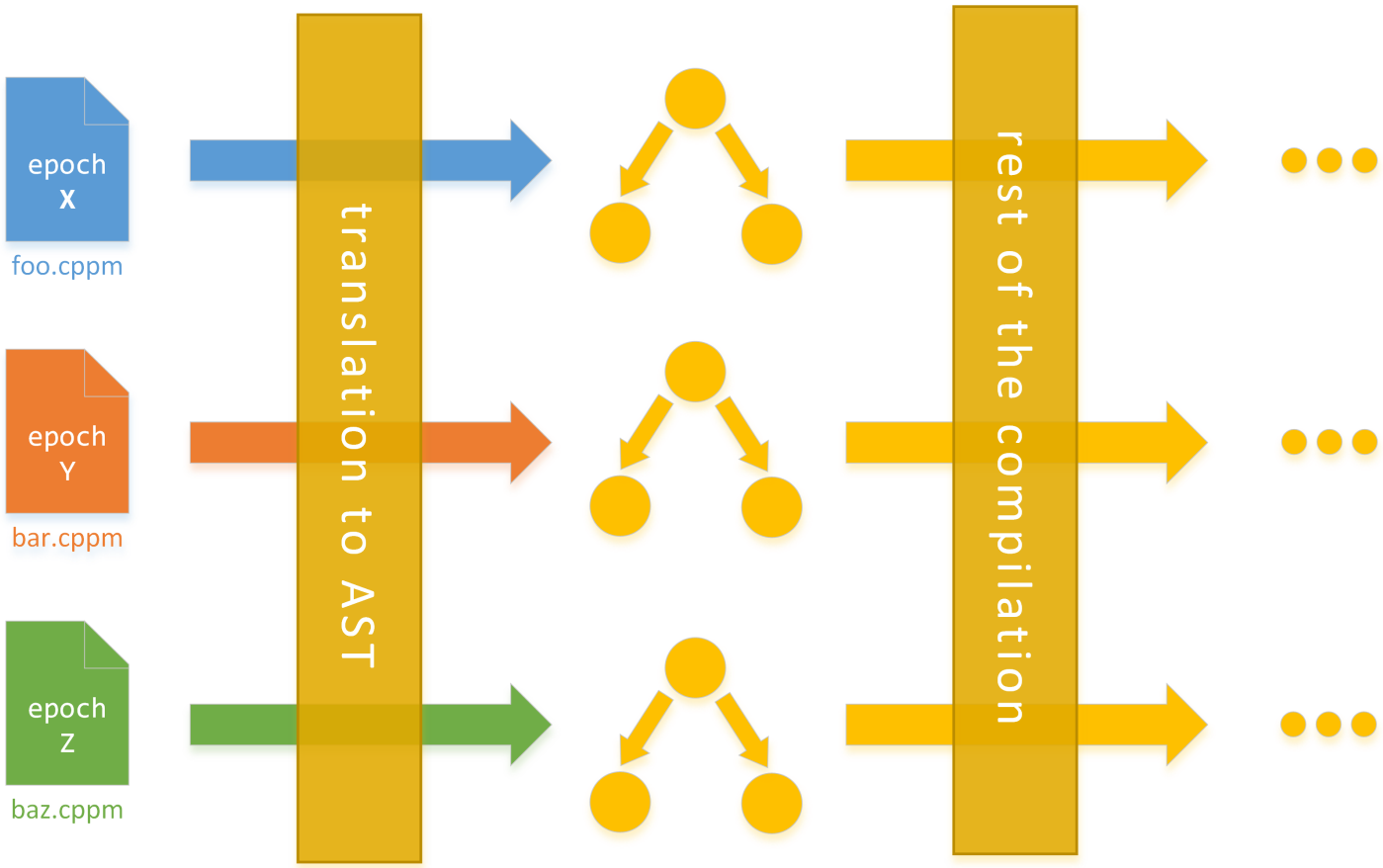
Modules can seamlessly import and consume other modules targeting different epochs, implying that multiple epochs can coexist as part of the same project without compatibility issues, and that a project can be gradually migrated to a newer epoch on a per-module basis.

```
module ParticleRendering;

// OK, even if the current module doesn't use `epoch 2023`
import ParticleMovement;

export void render(const Particle&);

void renderExample()
{
    Particle p{};

    // OK, this module allows implicit conversions
    move(p, 3.42, 2.49);

    render(p);
}
```

5 Mechanism

Adding epochs to C++ requirs one extra step in the compilation process: different modules targeting different epochs must be normalized to the same intermediate format (e.g. AST). The required machinery would be similar to today's standard `-std=` switches.

The diagram below shows three modules targeting three different epochs being compiled together as part of the same project:



5.1 Epoch declarations

— *Epoch-declarations* have the following form

> *epoch-declaration:*
>
> epoch *epoch-literal*;
>
> *epoch-literal:*
>
> 2023

and allow developers to opt into the changes provided by a specific epoch.

— Every *module* can optionally have a *epoch-declaration* before its *module-declaration*.

— If the *epoch-declaration* is present, code inside the *module purview* must obey the rules of the chosen epoch.

— **Definition**: a module *targets* epoch `X` if it contains an `epoch X;` *epoch-declaration*.

5.2 Epoch rules

— **Definition**: a module targeting epoch `X` is said to *obey* epoch `X` if it is well-formed according to the rules specified for epoch `X`.

— If a module must obey the rules of a given epoch, the way it is parsed might differ depending on the epoch.

  — This implies that the same code, under different epochs, might be parsed to sligthly different ASTs. Furthermore, the same code might be ill-formed under a set of epochs, and well-formed under others.

— The C++ standard will provide a set of rules for each epoch. While processing a module which opted into a particular epoch, compilers must follow those rules.

5.3 Epoch compatibility

— Modules can import and consume other modules targeting different epochs. The restrictions of an epoch only apply to the source code where the module is defined, not to the source code of the importer.

  — This is true even in the case of *templates*, as modules can export template definitions in a normalized metadata format.

— Multiple epochs can coexist as part of the same project without compatibility issues.

5.4 Epoch release cycle

— Every new C++ standard might introduce a new epoch, where the *epoch-literal* is a four-digit representation of the year in which the standard is released.

— Not all the features introduced in a new standard `X` must be introduced under the epoch `X` - some of them can become immediately available to users compiling against `X` (without altering source code).

— The intention is that breaking changes will very likely be protected behind epoch `X`, while pure additions to the language will likely not.

— Projects can also gradually migrate to use epoch-specific changes by opting-in on a per-module basis. Note that migration is and will never be mandatory.

— New modules introduced in a project can and should target the latest epoch.

6 Example: *uninitialized variables*

A common bug that epochs would help avoid is the misuse of uninitialized variables. C++ makes it extremely easy for developers to accidentally forget to initialize a variable:

```
auto countAndProcess(CSVStream& csvStream)
{
    std::string res;
    std::size_t cnt;

    while (csvStream)
    {
        res += process(csvStream.next());
        ++cnt; // Undefined behavior
    }

    return std::pair{result, count};
}
```

As an example, epoch `X` could require a more explicit syntax to define uninitialized variables, which would ensure the written code matches the developer's intentions:

Before	After

`module Example;`
 	
`int main()`
`{`
 	
`  int i;  // OK, uninitialized`
`  std::string s; // OK, default-constructed`
`}`
	`epoch X;`
	`module Example;`
	
	`int main()`
	`{`
	`  ~~int i;~~  // Compilation error`
	`  int i = void;  // OK, uninitialized`
	`  std::string s; // OK, default-constructed`
	`}`

The above table shows how modules targeting epoch `X` would require a more explicit syntax to define uninitialized variables. Writing `int i;` as part of a module targeting epoch `X` would result in the module not obeying the epoch's rules, thus being ill-formed and resulting in a compilation error.

Note that the `= void` syntax would be subject to bikeshedding, and that the entire idea is just an example of what epochs could do. This paper is not proposing any epoch rule - it is only proposing the mechanism itself.

7 Design Decisions

Epochs were designed with the following goals and principles in mind:

1. Provide a mechanism to improve and simplify the language while retaining backward and forward compatibility.
2. Prevent the proliferation of dialects.
3. Ensure that source code readers easily understand the effects of an epoch.
4. Allow the language to evolve without drastically changing the way it behaves or looks.

5. Allow graceful migrations between standards and epochs of arbitrarily-sized code bases.

6. No effect on ABI whatsoever.

(1) has already been explained. This section will take a closer look at the remaining points.

7.1 Dialects

One of the biggest concern with source-level switches that alter the meaning of code is that a plethora of slightly different dialects will profilerate in the C++ community.

Epochs are carefully designed to avoid this problem, as they do not provide many small tunable “knobs” - they instead provide a **single**, **linear monotonically increasing** sequence of language flavors. **Modules can target one and only one epoch in particular**, and each epoch builds on top of the previous one.

Additionally, epochs would only be added to the language simultaneously with a new standard release, and epoch-specific changes would be subject to the same scrutiny of any other language change.

7.2 Understandability

An argument against epochs is that the isolation provided by modules could allow the committee to simply apply breaking changes to a new standard, as users would be able to independently compile different modules against different standards and still link them together.

While the argument is deeply flawed under multiple aspects, the approach of using compiler switches has one massive drawback: **developers would not be able to understand what the meaning of C++ code without additional build-related context**. Naming conventions or comments would be required to demystify what a module allows/disallows or changes from others.

The presence of an *epoch-declaration* makes immediately obvious to reader what the meaning of the code is.

7.3 Gradual evolution

The mechanism described in this paper could theoretically allow the introduction of a module-level switch for a novel language compatible with C++. Such language would create an irreparable fracture in the community, and would incredibly complicate teaching, understandability, and user-friendliness of C++.

One of the main principles of epochs is that **C++ should still look like C++**. Since every epoch-specific change would still need to reach consensus in the committee (whose members understand the importance of keeping the language consistent and the community together), this principle will not be violated.

7.4 Graceful migration

While the committee tries to minimize breaking changes between standard, sometimes they are introduced (often with good reasons), resulting in a migration cost which can be massive for large-scale organizations.

Ensuring introduction of breaking changes in a new standard only as part of an epoch would greatly enhance migration for companies and individuals, as every project would be able to immediately and safely switch to a new standard to benefit from new features, while gradually converting existing modules to modernize and increase confidence in codebase's robustness.

Additionally, upgrading a module from epoch `X` to epoch `X + 1` should be easy - a good guideline would be to ensure that an automatic tool (possibly provided as part of the compiler) can perform the migration.

7.5 No ABI impact

It is a strict requirement that epochs must not affect ABI. Epochs will not introduce any change that results in ABI breakage - their role is to sligthly affect how source code tranforms to an AST and whether it is considered well-formed or ill-formed.

8 Possible use cases

This section contains various possible use cases for epochs, only for illustrative purposes (not being proposed as part of this paper). Furthermore, the list is not exhaustive. The general goals of the use cases reported below are:

— Make C++ safer by default, without reducing its power. This often implies requiring the user to more precisely state their intention in code, or to require the user to opt-in to a less safe construct (in contrary to today's situation where users generally have to opt-in into safer constructs).

— Make C++ easier to teach, learn, and use by reducing its complexity. This implies the removal or repurposing of existing features in order to reduce the number of choice that an user has to perform a particular task, nudging them towards a safe and homogenous solution.

An important side benefit of the aforementioned goals is that C++ code becomes easier to read and to debug.

8.1 Remove obsolete features

Use of older features that have a more modern counterpart could be forbidden in order to reduce the size and complexity of the language and encourage writing better code. `typedef` and C-style arrays are two examples:

Before	After

`module Example;`
 	
`int main()`
`{`
`  int a0[]{1, 2}; // OK`
`  std::array<int> a1{1, 2}; // OK`
 	
`  typedef int I0; // OK`
`  using I1 = int; // OK`
`}`
	`epoch X;`
	`module Example;`
	
	`int main()`
	`{`
	`  ~~int a0[]{1, 2};~~ // Compilation error`
	`  std::array<int> a1{1, 2}; // OK`
	
	`  ~~typedef int I0;~~ // Compilation error`
	`  using I1 = int; // OK`
	`}`

8.2 Introduce new keywords or rename existing ones

Introducing new keywords has always been difficult due to possible name collisions with existing code. A notable example is the addition of `co_await`, `co_yield`, and `co_return`, presenting an unusual (and universally disliked) `co_` prefix to avoid breaking older code.

Epochs would provide a safe context where new keywords can be introduced without worrying about backward compatibility, as no modules targeting an unreleased epoch can exist. Similarly, existing keywords could be renamed.

Before	After

`module Example;`
 	
`generator<int> getNumber()`
`{`
 	
`  co_yield 42; // OK`
`}`
	`epoch X;`
	`module Example;`
	
	`generator<int> getNumber()`
	`{`
	`  ~~co_yield 42;~~ // Compilation error`
	`  yield 42;  // OK`
	`}`

A drawback of this approach would be that - for example - a class exposing a `virtual` member function named `await` could not be extended in `epoch X`. This, however, is not a problem in practice due to the small likelihood of such occurrences and thanks to the fact that targeting a epoch `X` in a module is not mandatory. Introducing a “keyword escape” syntax could also be a possible solution, albeit unnecessarily complicated according to the taste of this paper’s author.

Other ideas regarding keywords include:

— Preventing the use of `class` as a template parameter.

— Remove english alternatives (`and`, `or`, etc.) to boolean operators.

— Enforce english alternatives to boolean operators for boolean expressions, enforce `&` and `&&` for references.

8.3 Enforce use of `nullptr`

Modern code should use `nullptr` instead of `0` or `NULL` to maximize readability and prevent suprising overload resolution outcomes between pointer and integral types. An epoch could forbid the use of the integer literal `0` and of the macro `NULL` in a context where a pointer is required:

Before	After

`module Example;`
 	
`void foo(long); // (0)`
`void foo(int*); // (1)`
 	
`int main()`
`{`
`  int* p0 = 0; // OK`
`  int* p1 = NULL; // OK`
`  int* p2 = nullptr; // OK`
 	
`  foo(0);  // Ambiguous?`
`  foo(NULL);  // Ambiguous?`
`  foo(nullptr); // Calls (1)`
`}`
	`epoch X;`
	`module Example;`
	
	`void foo(long); // (0)`
	`void foo(int*); // (1)`
	
	`int main()`
	`{`
	`  ~~int* p0 = 0;~~  // Compilation error`
	`  ~~int* p1 = NULL;~~  // Compilation error`
	`  int* p2 = nullptr; // OK`
	
	`  foo(0);  // Calls (0)`
	`  ~~foo(NULL);~~  // Compilation error`
	`  foo(nullptr); // Calls (1)`
	`}`

8.4 Enforce use of `break` or `fallthrough` in switches

An epoch could safely introduce a new `fallthrough` keyword and require each `case` in a `switch` statement to either end with `break;` or `fallthrough;` in order to prevent bugs and aid readability.

Before

```
        module Example;

        void example(int
choice)
        {
            switch(choice)
            {
                case 0:

                case 1:

something();

                case 2:

something();
                    break;
                case 3:

something();

            };
        }
```

After

```
        epoch X;
        module Example;

        void example(int choice)
        {
            switch(choice)
            {
                case 0:
                    fallthrough; //
Required
                case 1:
                    something();
                    fallthrough; //
Required
                case 2:
                    something();
                    break;
                case 3:
                    something();
                    break; // Required
            };
        }
```

Notably, this change would have prevented a severe performance bug at Bloomberg caused by forgetting a `break;` statement.

8.5 Requiring explicit syntax for uninitialized variables

Discussed above.

8.6 Preventing implicit conversions of fundamental types

Discussed above.

8.7 Replace `std::initializer_list` with a better alternative

Roughly speaking, `std::initializer_list<T>` is syntactic sugar over a `const T[]`, which does not allow its elements to be moved.

An epoch could introduce a new `std::movable_initializer_list` type which would be designed to work nicely with move semantics and with the previous `std::initializer_list`, and change the meaning of braced initialization to instantiate the new type instead of the old one.

Such a change would allow existing code to work, while enabling new code to take advantage of a more powerful `std::initializer_list` alternative without requiring a brand new initialization syntax.

Before	After

module Example;
 	
int main()
{
// OK
// `l0`` is `std::initializer_list<int>`
auto l0 = {1, 2, 3, 4};
 	
// Compilation error
 	
~~std::vector<std::unique_ptr<Foo>>> v{~~
 	
~~std::make_unique<Foo>(0),~~
 	
~~std::make_unique<Foo>(1)~~
~~};~~
}
	epoch X
	module Example;
	
	int main()
	{
	// OK
	// `l0` is a `std::movable_initializer_list<int>`
	auto l0 = {1, 2, 3, 4};
	
	// OK, `std::vector` has a new constructor from
	// `std::movable_initializer_list<int>`
	
	std::vector<std::unique_ptr<Foo>>> v{
	
	std::make_unique<Foo>(0),
	
	std::make_unique<Foo>(1)
	};
	}

8.8 Improve `std::initializer_list` and *uniform initialization* interactions

Currently, variable initialization can subtly and massively change meaning depending on what syntax is used. As an example, `std::vector<int>{4, 4}` is wildly different from `std::vector<int>(4, 4)`. Many agree that this behavior is problematic (especially in template definitions), and that it prevents developers from uniformly using curly braces everywhere, thus defeating the purpose of *uniform initialization*.

An epoch could introduce a new unambiguous syntax to invoke `std::initializer_list` constructors, which as an example here will be a double set of curly braces. With this new syntax, multiple approaches could be taken:

— Introduce ambiguity errors when non-`std::initializer_list` constructors would be viable alongside `std::initializer_list` ones:

Before	After

`module Example;`
 	
`std::vector<int> v0(4, 4); // OK, [4, 4, 4, 4]`
`std::vector<int> v1{4, 4}; // OK, [4, 4]`
	`epoch X`
	`module Example;`
	
	`std::vector<int> v0(4, 4); // OK, [4, 4, 4, 4]`
	~~`std::vector<int> v1{4, 4};`~~
	`// Compilation error: ambiguous`
	`std::vector<int> v2{{4, 4}}; // OK, [4, 4]`

— Only make invocation of `std::initializer_list` constructors possible through the new syntax, and make a single pair of curly braces unambiguously target any constructor that does not take `std::initializer_list`. This change could also be applied to aggregate and array initialization.

8.9 Remove outdated initialization syntax

C++ currently supports many different initialization syntaxes, including:

— `int i0 = 0;`

— `int i1(0);`

— `int i2{0};`

— `int i3 = {0};`

An epoch could reduce the number of possibilities and the complexity of the language by forbidding a subset of the existing approaches:

Before	After

`module Example;`
 	
`int i0 = 0;  // OK`
`int i1(0);  // OK`
`int i2{0};  // OK`
`int i3 = {0}; // OK`
	`epoch X`
	`module Example;`
	
	~~`int i0 = 0;`~~ `// Compilation error`
	`int i1(0);  // OK`
	`int i2{0};  // OK`
	~~`int i3 = {0};`~~ `// Compilation error`

This idea, combined with a more poweful `std::initializer_list` alternative that plays nicely with *uniform initialization*, could lead to a truly unique universal initialization syntax.

8.10 Enforce `explicit` or `implicit` for constructors

`explicit` constructors should be preferred to implicit ones in order to avoid suprising conversions.

An epoch could introduce a new `implicit` keyword, and require either `explicit` or `implicit` to be specified when defining a constructor. This would encorage developers to use `explicit` and force them to think about whether they want implicit conversions for their types or not.

Before	After

`module Example;`
 	
`class Foo`
`{`
`  // OK, implicit`
`  Foo(int);`
 	
`  // OK, explicit`
`  explicit Foo(Bar);`
`};`
	`epoch X;`
	`module Example;`
	
	`class Foo`
	`{`
	`  // Compilation error`
	`  ~~Foo(int);~~`
	
	`  // OK, implicit`
	`  implicit Foo(int);`
	
	`  // OK, explicit`
	`  explicit Foo(Bar);`
	`};`

8.11 Enforce `const` or `mutable` for variables

`const` should be used whenever possible to reduce cognitive overhead introduced by mutable state, to avoid uninitialized variables, and to prevent bugs.

While making variable definitions `const` by default and allowing usage of `mutable` to suppress constness (like in lambda expressions) might seem like a sensible idea at first, it does violate the principle that C++ should look like C++ and that new epochs should not drastically change the meaning of familiar code.

A more sensible approach would be requiring either `const` or `mutable` to be used whenever a variable is defined. This would encourage developers to use `const` (due to the verbosity of `mutable`) and force them to make a conscious decision about mutability, without changing the meaning of existing C++ code.

Before	After

`module Example;`
 	
`void foo(float f0, // OK, mutable`
`const float f1) // OK, immutable`
`{`
`int i0; // OK, mutable`
 	
`const int i1 = 42; // OK, immutable`
`}`
	`epoch X;`
	`module Example;`
	
	`void foo(~~float f0,~~ // Compilation error`
	`mutable float f0, // OK, mutable`
	`const float f1) // OK, immutable`
	`{`
	`~~int i0~~; // Compilation error`
	`mutable int i0; // OK, mutable`
	`const int i1 = 42; // OK, immutable`
	`}`

8.12 Enforce uncontroversial Core Guidelines

The Core Guidelines[4] project was created in order to provide the C++ community with a set of guidelines that promote safe and effective usage of the C++ language and standard library. The mere existence of these guidelines suggests that there is something problematic with C++: a language should not require its users to peruse a document which explains how to avoid various pitfalls in order to be used correctly.

Epochs would allow the least subjective and most uncontroversial guidelines to be enforced by the compiler, aiding newcomers and experts alike. For guidelines which do not universally apply to all programs, opt-out syntax could be provided. Here is a non-exhaustive selection of guidelines that could be considered to be introduced in the language as part of an epoch:

— F.55: Don't use `va_arg` arguments (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#F-varargs)

  - The use of `va_arg` could be forbidden or confined to an `unsafe` block.

- C.8: Use `class` rather than `struct` if any member is non-public (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rc-class)
  - Could be enforced starting from a particular epoch.
- C.46: By default, declare single-argument constructors explicit (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rc-explicit)
  - Discussed above.
- C.128: Virtual functions should specify exactly one of `virtual`, `override`, or `final` (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rh-override)
  - Could be enforced starting from a particular epoch.
- C.127: A class with a virtual function should have a virtual or protected destructor (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rh-dtor)
  - Could be enforced starting from a particular epoch.
- Enum.3: Prefer class enums over “plain” enums (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Renum-class)
  - The use of C-style enums could be forbidden or confined to an `unsafe` block.
- R.6: Avoid non-`const` global variables (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rr-global)
  - The use of non-`const` globals could be forbidden or confined to an `unsafe` block.
- R.10: Avoid `malloc()` and `free()` (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rr-mallocfree)
  - Their use could be forbidden or confined to an `unsafe` block.
- R.11: Avoid calling `new` and `delete` explicitly (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rr-newdelete)
  - Their use could be forbidden or confined to an `unsafe` block.
- R.14: Avoid `[]` parameters, prefer `span` (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rr-ap)
  - Their use could be forbidden or confined to an `unsafe` block.

- ES.25: Declare an object `const` or `constexpr` unless you want to modify its value later on (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Res-const)
  - Discussed above.
- ES.47: Use `nullptr` rather than `0` or `NULL` (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Res-nullptr)
  - Discussed above.
- CP.8: Don't try to use `volatile` for synchronization (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rconc-volatile)
  - Use of `volatile` could be forbidden and replaced by functions described in P1382[5].
- Con.1: By default, make objects immutable (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rconst-immutable)
  - Discussed above.
- Con.2: By default, make member functions const (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rconst-fct)
  - Could be enforced by a mechanism similar to the choice between `mutable` and `const` previously discussed for variables.

8.13 Introduce a placeholder keyword

A common request of C++ users is the addition of a special placeholder name keyword which could be used to instantiate variables with scope lifetime that have a unique unutterable name - useful for types like `std::scoped_lock`. There have been attempts to do this in the past, but most were shut down due to the possibility of name collisions between the placeholder syntax and existing symbols.

Epochs would elegantly solve this problem by giving _ the special meaning of *"unique and anonymous identifier"*.

8.14 Make functions `[[nodiscard]]` by default

Commonly, function returning a value require the caller to inspect their result even if they have side-effects. Most functions with a non-`void` return type should therefore be marked with `[[nodiscard]]`. Unfortunately, the verbosity of the attribute discourages a large number of developers from doing that.

A more sensible default would be for all functions to implicitly behave as if they were marked with `[[nodiscard]]`, and to provide a `[[discardable]]` attribute which could be used to clearly mark functions whose return value is not always significant. Epochs would make this change possible.

8.15 Enforce one declaration style

C++ currently allows developers to choose between different declaration styles for both functions and variables:

— `int i = 0;` versus `auto i = int{0};`

— `int foo();` versus `auto foo() -> int;`

Disallowing one of these choices from a particular epoch onwards might increase the consistency of future C++ code, reduce analysis paralysis, and possibly improve readability. Side benefits of forcing `auto` for variable declarations include resolving the “most vexing parse” issue and preventing definition of uninitalized variables.

8.16 Standard Library Changes

Since epochs will not break ABI, it is easy to believe that the standard library could not benefit from them, which is far from true. One possible way of removing outdated and dangerous standard library facilities would be to forbid some symbols from being usable, without actually removing the facility. This could be controlled with some sort of annotation:

```
namespace std {

template <typename T>
class optional {
public:
    [[accessible_until_epoch(X)]]
    const T& operator*() const;

    [[accessible_since_epoch(X)]]
    const T& unsafe_get() const;
};

}
```

The above example means that any attempt to invoke `std::optional<T>::operator*()` from a module targeting epoch `X` would result in a compilation failure, even though the member function exists. Similarly, `std::optional<T>::unsafe_get()` would only be available in modules targeting epoch `X`.

This approach would allow the committee to "blacklist" certain interfaces/types and encourage the use of others without breaking backward or ABI compatibility.

Another area of research might be changing the meaning of a library symbol (e.g. `std::vector` becomes an alias for `std::vector2`).

9 Frequently Asked Questions

> In practice, what problems are solved by epochs?

Well-researched problems that affected large corporations, such as some described in the *"Curiously Recurring C++ Bugs at Facebook"* CppCon 2017 talk[6] and corresponding `r/rust` thread,[7] will be either solved or mitigated by epochs. From that list: bound-safe accesses could become the default for standard

containers; the behavior of `std::map::operator[]` could be changed to avoid the creation of default elements (possibly by blacklisting this API in an epoch and providing a safer one).; and the use of `volatile` could be forbidden.

Another issue that the author of this paper has personally experienced is the pain of migrating to a newer standard for a large corporation. Removal of standard library entities (such as `unary_function` and `binary_function` in the case of C++17) cause the inability for many legacy projects to use a new standard without manual intervention. Some companies, like Bloomberg, use a system where the entire company's codebase has to compile on the same toolchain and flags in order to guarantee consistency and compatibility between different teams' projects. Having to perform manual changes throughout the entire company to finalize a migration means that most of the teams will be stuck on an older standard until legacy code is needlessly cleaned up. Using epochs to perform removals and breaking changes would allow such migrations to be performed gradually, and allow non-legacy projects to immediately take advantage of newer standards without being blocked by legacy software.

Finally, the language would become much more friendly and accessible to newcomers. This is important to ensure the growth of the language, to simplify the training and learning process, and to maximize the chances of building a diverse community of talented developers who want to use C++ and participate in its evolution. The author of this paper has delivered C++ training to hundreds of people of different skill levels, and strongly believes that the complexity of topics such as variable initialization could be eradicated by using a mechanism like epochs. After explaining how to enable the latest epoch to students, the training could focus on a safe and logical subset of the latest standard that does not provide needlessly varied and complicated choices. Furthermore, students attempting to use unsafe constructs that they learned from C or poor C++ training material would be stopped by the compiler before introducing undefined behavior into their code.

> Why not provide fine-tuned knobs to enable/disable/tweak various features instead of arbitrarily large epochs?

While some people believe that fine-tuned knobs (multiple independent flags at the beginning of a source file to control the behavior/accessibility of different language/library constructs) would ease migration from an epoch to another, they fail to understand the implication of such mechanism. Having this freedom would create an incredible amount of complexity as every single module could behave in a slightly different but significant way from another, and the only way for a developer to deal with that would be to keep all the flags

given at the beginning of the file in their mind. This cognitive overhead defeats the purpose of epochs and is exacerbated when considering how often developers read multiple files simultaneously, which might have completely different settings.

Providing a linear and incremental model for epochs is essential to avoid the aformentioned complexity and cognitive overhead, and it also ensures that the language evolves in a single direction dictated by consensus between the community and the committee. Concerns regarding ease of migration are easily dismissed by the fact that one of the guiding principles of epochs is the fact that migrations should be easy and automatable, and - most importantly - that no one should feel forced to migrate to a newer epoch. Non-breaking language and library additions will not be confined to epochs and, while it increases the safety and readability of a module, targeting a newer epoch it is not a necessity.

> How do I deal with C headers or old C++ headers?

Epochs are designed around modules, which are expected to become the norm for C++ development in the near future. Conversion or wrapping of headers into modules is the preferred approach to solve any potential incompatibility introduced by targeting an epoch. If conversion is not possible, headers can be imported as “header units”, which would help with consuming them from modules that target a particular epoch. If neither conversion or wrapping is possible, and if header units do not prevent incompatibilities, then the only drawback is that a particular header cannot benefit from the changes introduced in a new epoch. As mention in the answer above, this is not a big deal - not everything has to target the latest epoch.

> I cannot migrate from epoch X to epoch Y, but I really need a feature added in standard Y. What can I do?

If the feature is considered a breaking change and confined to epoch Y, you will have to figure out a way to migrate or to refactor your code in such a way that epoch Y becomes accessible in the code path where it is required. If the feature is not considered a breaking change, it will be retroactively available in older epochs with the release of standard Y. A real example of that happening comes from Rust, with the backporting of a 2018-specific feature to the 2015 edition.[8]

> What would the ISO C++ standard document look like with the addition of epochs?

This has not yet been researched. Intuitively, with only one or two epochs, epoch-specific behavior could be specified as part of the existing wording of features. If a considerable number of epoch is added, then providing per-epoch wording might be a better solution.

> Why do we need epochs? Can't we just change the standard targeted by a module by using compiler switches?

Having only a single level of choice for the meaning of source code provides a large number of problems which epochs try to address. Imagine if switching from `-std=c++XX` to `-std=c++YY` introduced significant breaking changes or changed the meaning of some existing constructs: it would now be impossible to understand what the behavior of C++ source code is just by reading it, as it would depend on compilation flags. While we do have this problem today, its impact is small as the number of breaking changes introduced with every standard is miniscule. Epochs aim to allow the committee to clean up and polish the language by preventing this kind of confusion thanks to the *epoch-declaration* at the beginning of a module file.

Furthermore, using compiler switches instead of epochs makes migration and building a lot harder, especially where header files are present. Codebases will have to selectively choose what source files are compiled towards a particular standard, and what source files are compiled towards another. Header files would require to either be duplicated to support breaking changes introduced in different standards or to avoid using any construct which can change meaning, severely limiting what - for example - template definitions can use.

Introducing more breaking changes without a mechanism like epochs would be a disaster for the C++ language and community.

> Wouldn't this feature make it hard to copy-paste code between different epochs?

It is possible that copying code from a module targeting an older epoch and pasting it into a module targeting a newer one would result in ill-formed code. However, according to the "easy migration" principle, making the changes required to attain conformity with the newer epoch will either be easy or automatic.

While the inability of copy-pasting code is indeed a drawback of epochs, it is one very small price to pay for a mechanism which would enable C++ to move towards a safer and more modern direction.

10 Existing practice

Various languages adopt mechanisms similar to the proposed epochs with similar goals in mind:

— Rust's "Editions" feature[9] is the most notable example, as it inspired this proposal and as its implementation/rationale are quite similar to what is being proposed in this paper.

— CMake provides the `cmake_minimum_required` construct[10] which can be provided at the beginning of a `CMakeLists.txt` file in order to allow the file to be used only with a particular version of CMake. This ensures that any addition or breaking change provided in a new version is intentionally desired by the `CMakeLists.txt` file.

— The C# language has recently introduced a way to change one of the language's fundamental (yet very dangerous) defaults: nullable variables[11]. By adding the `#nullable disable` annotation at the beginning of a source file, the meaning of code is drastically changed to avoid bugs and force people to write more explicit and safer code.

— The PHP community is pushing forward a proposal for "P++,"[12] which would allow developers to provide a `<?p++?>` directive at the beginning of source files to disallow the usage of outdated and dangerous constructs. It is very similar in spirit and goals to this proposal.

(The list above is not exhaustive.)

11 Bikeshedding

The term "epoch" is subject to bikeshedding. Here are some other potential names:

— Edition

— Dialect

— Flavor

— Generation

— Version

— Standard

— Ruleset

12 Acknowledgments

Thanks to Joshua Berne and Corentin Jabot for providing feedback on an early draft of this proposal.

13 References

---

1. https://en.wikipedia.org/wiki/Analysis_paralysis (https://en.wikipedia.org/wiki/Analysis_paralysis)↩

2. https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rh-override (https://github.com/isocpp/CppCoreGuidelines/blob/master/CppCoreGuidelines.md#Rh-override)↩

3. http://wg21.link/p0709 (http://wg21.link/p0709)↩

4. https://github.com/isocpp/CppCoreGuidelines (https://github.com/isocpp/CppCoreGuidelines)↩

5. http://wg21.link/p0709 (http://wg21.link/p0709)↩

6. https://www.youtube.com/watch?v=lkgszkPnV8g (https://www.youtube.com/watch?v=lkgszkPnV8g)↩

7. https://old.reddit.com/r/rust/comments/cq9rco/cppcon_2017_curiously_recurring_c_bugs_at_facebook/ (https://old.reddit.com/r/rust/comments/cq9rco/cppcon_2017_curiously_recurring_c_bugs_at_facebook/)↩

8. https://github.com/rust-lang/rust/pull/60932/ (https://github.com/rust-lang/rust/pull/60932/)↩

9. https://doc.rust-lang.org/edition-guide/editions/index.html (https://doc.rust-lang.org/edition-guide/editions/index.html)↩

10. https://cmake.org/cmake/help/latest/command/cmake_minimum_required.html (https://cmake.org/cmake/help/latest/command/cmake_minimum_required.html)↩

11. https://devblogs.microsoft.com/dotnet/try-out-nullable-reference-types/ (https://devblogs.microsoft.com/dotnet/try-out-nullable-reference-types/)↩

12. https://wiki.php.net/pplusplus/faq (https://wiki.php.net/pplusplus/faq)↩